

敏感区域入侵目标智能识别系统

摘要

针对当前边缘智能监控领域高度依赖海外 AI 芯片、存在技术卡脖子与数据隐私泄露的行业痛点，本作品以瑞芯微国产 RK3588 八核嵌入式 SoC 为硬件底座，自主研发一套全链路国产化实时目标检测系统。系统打通 PC 模型训练 RKNN 模型量化适配—> Python 多线程流水线推理—> HDMI 本地实时显示完整工程链路，选用 YOLOv8n 轻量化检测网络(3M 参数)，解决嵌入式端推理卡顿、延迟过高、多路视频兼容难题。硬件采用 ELF2(RK3588)开发板作为主控，使用 RTSP 网络监控摄像头作为视频输入源;软件层面基于 RKNNToolkit2 完成 pt>ONNX>RKNN 模型格式转换与 FP16 量化，充分利用 RK3588 内置 6TOPSNPU 硬件加速;针对官方单线程推理框架实时性差的缺陷，自主设计基于 Pythonthreading 的多线程流水线架构，将视频帧读取与 NPU 推理分离到独立线程并行执行;依托 ffmpeg 子进程实现 RTSP 视频流低延迟解码，所有视频数据本地离线处理，无需上传云端，保障数据安全。系统配套 argparse 命令行参数，无需修改代码即可切换摄像头地址、调整置信阈值、更改显示分辨率。经完整实测:640X480@30fps 视频输入下稳定输出 10~14FPS 推理速度，自建无人机与行人数据集 mAP@0.5 达 0.92，端到端检测延迟约 1 秒(受 ffmpegRTSP 解码管线固有延迟影响)，CPU 占用率低于 40%。作品可直接落地智能园区安防、野外无人监测、工业轻量化质检、智慧交通等场景，彻底摆脱英伟达、海外专用 AI 芯片依赖。全部研发工作由大二本科学生独立完成，覆盖深度学习训练、嵌入式 Linux 开发、Python 多线程编程、多媒体处理多学科交叉内容，验证了国产 RK3588 芯片在边缘 AI 领域完整落地可行性，国产化嵌入式视觉设备提供标准化工程解决方案。

第一部分 作品概述

1.1 功能与特性

本系统实现国产化嵌入式设备端全实时目标检测，核心功能分为四大模块：

1. RTSP 网络摄像头视频采集：通过 `ffmpeg` 子进程以低延迟模式拉取局域网 RTSP 监控摄像头视频流，硬件解码为原始 BGR 帧；

2. NPU 加速目标推理：RKNN 量化 YOLOv8n 模型，依托 RK3588 NPU 完成硬件加速检测，输出目标坐标、类别、置信度；

3. Python 多线程流水线优化：自研 `reader-infer-display` 多线程架构，解决单帧串行推理造成的画面卡顿与高延迟问题；

4. 本地 HDMI 全屏显示：推理结果绘制标注框与敏感区域后通过 `OpenCV` 全屏显示，支持鼠标交互绘制多边形敏感区域与入侵报警。

系统全部图像处理、推理任务由芯片硬件单元加速，CPU 仅做数据搬运与绘制；采用 `argparse` 命令行参数管理模型路径、NMS 阈值、RTSP 地址、显示分辨率等参数；整机离线独立运行，原始监控数据不对外传输，从硬件底层规避隐私泄露风险。整体轻量化、低功耗，无云端依赖，适配各类无公网、数据敏感边缘应用场景。

1.2 应用领域

1. 园区、厂区智能安防：厂区、小区网络摄像头实时识别人、车辆，自动标记入侵目标，减少人工值守，数据本地存储不上云，规避监控数据泄露风险；

2. 野外无人值守监测：水库、山林、光伏基站等偏远点位，依托 RTSP 摄像头本地检测，仅推送标注后视频，大幅节省远程带宽；

3. 轻量化工业视觉质检：小型生产线通过 RTSP 网络摄像头离线识别工件、杂物，无需搭建云端算力平台，降低设备采购成本；

4. 智慧交通路边感知：路口低功耗嵌入式设备识别行人、非机动车，采集轻量化交通结构化数据；

5. 校园、商超客流统计：边缘本地统计人员数量，原始画面不上传，满足公共场所隐私合规要求。

6. 航空安全跑道监控：机场跑道周边部署 RTSP 网络摄像头，实时检测闯入跑道的人员、车辆或异物，触发本地入侵报警，数据离线处理不上云，满足机场

安全管控的严格合规要求。

整套方案基于国产 RK3588 芯片搭建，无海外核心器件依赖，适配政府、工厂、园区等对信息安全有硬性要求的场景，具备极高落地价值。

1.3 主要技术特点

1.全国产化硬件平台：采用瑞芯微 RK3588 国产主控，CPU、NPU、多媒体编解码单元全部自研，规避海外芯片技术封锁；

2.YOLOv8 模型 RKNN 深度适配：基于 RKNN Toolkit2 完成 pt → ONNX → RKNN 模型量化适配，配置 mean=0、std=255 归一化参数，FP16 量化精度损失低于 1%，完成 FP16 量化方案，平衡精度与推理速度；

3.多路视频统一处理架构：一套程序兼容 RTSP 网络摄像头单一视频源，配合 ffmpeg 子进程低延迟解码，降低 CPU 开销；

4.自主多线程推理池创新：基于 Python threading 的多线程流水线架构，读帧线程与推理线程并行执行，解决单线程串行瓶颈，解决官方例程单线程高延迟缺陷；

5.多层硬件加速管线：ffmpeg 低延迟 RTSP 解码 + NPU 推理 + OpenCV 显示的轻量化流水线，合理分配 CPU 与 NPU 负载；

6.本地 HDMI 全屏检测显示：推理结果实时绘制并 OpenCV 全屏输出，支持键盘快捷键退出，交互式敏感区域绘制；

7.工程化配置解耦：argparse 命令行参数管理 RTSP 地址、模型路径、置信度阈值、显示分辨率等参数，无需修改代码即可快速切换场景。

1.4 主要性能指标

表 1 主要性能指标

测试指标	实测参数
主控芯片	瑞芯微 RK3588 (6TOPS NPU)
输入视频规格	640×480@30fps H.264 (RTSP 网络摄像头)
稳定推理帧率	10~14 FPS
检测精度 mAP@0.5	0.92 (自建数据集)
端到端检测延迟	约 1 秒 (含 ffmpeg 解码管线延迟)
整机 CPU 平均占用	35~40%
支持视频源	RTSP 网络监控摄像头

并发推理线程	2（读帧线程 + 推理线程）
显示方式	HDMI 全屏输出（cv2.imshow）

1.5 主要创新点

1.国产化全链路创新：完整实现从 PyTorch 模型训练到 RK3588 嵌入式部署国产闭环，摆脱英伟达 GPU、海外 AI 开发平台依赖，响应自主可控国家战略；

2.多线程流水线原创优化：针对 RKNN 官方单线程框架缺陷自主设计 Python 多线程流水线架构，读帧与推理并行执行，有效缓解画面卡顿，属于学生自主算法工程创新；

3.硬件加速推理管线：充分利用 RK3588 NPU 6TOPS 算力完成 YOLOv8n 模型硬件加速推理，配合 ffmpeg 低延迟 RTSP 解码，构建完整的端侧视觉流水线；

4.RTSP 网络摄像头通用工程方案：一套 Python 脚本兼容不同品牌 RTSP 网络摄像头，argparse 参数快速切换，具备良好工程复用与产品化价值。

1.6 设计流程

项目整体分为 6 个标准化研发阶段：①PC 端搭建 PyTorch 环境，基于自建无人机与行人数据集（11636 张）训练 YOLOv8n 模型，验证损失、精度指标收敛达标；②虚拟机搭建 RKNN Toolkit2 环境，完成 pt→ONNX→RKNN 模型转换与 FP16 量化调优；③ELF2 开发板烧写 Linux 系统，配置网络、ffmpeg、Python 依赖环境；④嵌入式软件开发：ffmpeg RTSP 低延迟解码、RKNN NPU 推理、Python 多线程流水线架构、敏感区域入侵检测；⑤HDMI 全屏显示与交互调试，argparse 命令行参数配置；⑥整机软硬件联调，完成 RTSP 网络摄像头单场景功能与性能量化测试。

第二部分 系统组成及功能说明

2.1 整体介绍

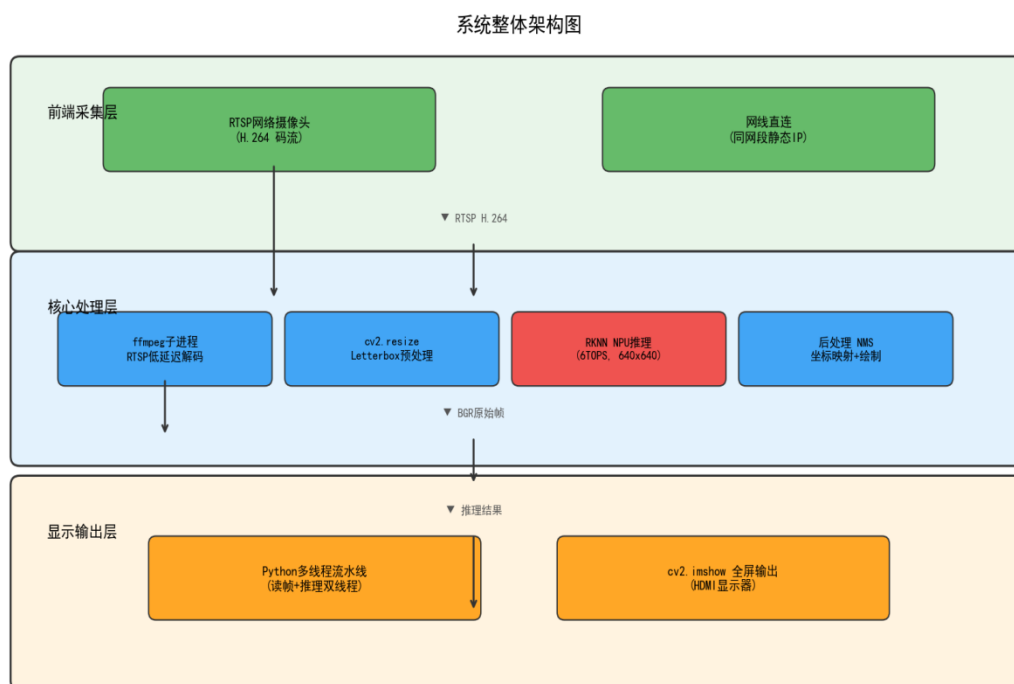


图 1 系统整体架构图

本系统整体架构分为前端视频采集模块、RK3588 核心处理模块、后端 HDMI 显示模块三层，三层数据单向流转、分工明确，系统总体框图如下：

1.前端采集模块：RTSP 网络摄像头通过网线直连开发板千兆网口，H.264 码流经由 RTSP 协议送入主控；2.核心处理模块：ffmpeg 子进程低延迟解码为 BGR 原始帧，Python 多线程流水线调度 RKNN NPU 推理，后处理绘制检测框与敏感区域；3.后端显示模块：OpenCV 全屏输出带标注的实时检测画面至 HDMI 显示器。数据完全本地闭环，不上传云端。

2.RK3588 核心处理模块（系统核心）：集成视频解码、图像预处理、多线程 NPU 推理、图像绘制、硬件编码、流媒体推送六大子单元，所有计算依托芯片硬件单元，CPU 仅做业务调度；

3.后端显示模块：OpenCV 调用 cv2.imshow 全屏输出实时检测画面至 HDMI 显示器，支持键盘快捷键（ESC/Q）退出。检测框、敏感区域、FPS 信息等全部在 Python 层绘制叠加。数据流转链路：原始视频帧 → ffmpeg 子进程 BGR 解码 → cv2.resize Letterbox 预处理 → NPU 推理 → 后处理 NMS

→ OpenCV 绘制 → HDMI 全屏显示。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

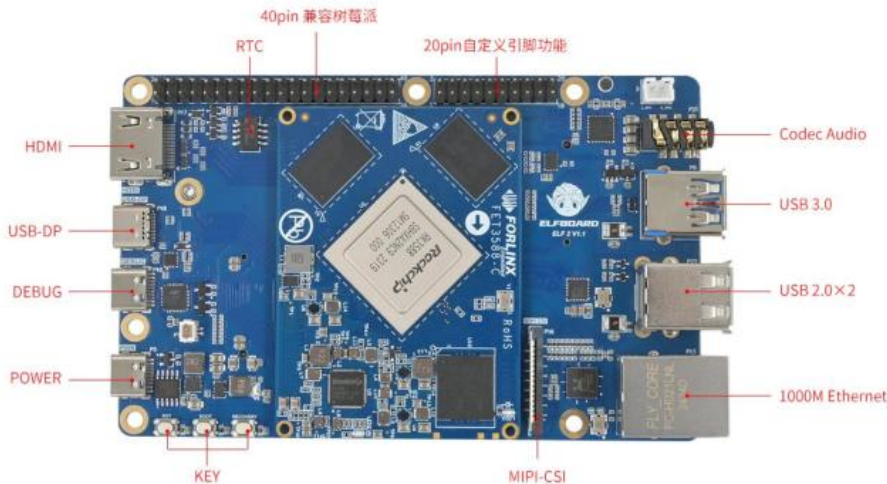


图 2 ELF2 开发板

主控硬件选用 ELF2（RK3588）嵌入式开发板，硬件核心参数：

4×A76+4×A55 八核处理器，独立 6TOPS NPU；外设配套：RTSP 网络监控摄像头、12V 直流稳压电源、HDMI 调试显示器、TTL 串口调试器、网线。硬件分工：开发板负责全部 AI 推理、视频解码与显示输出；网络摄像机通过网线直连开发板千兆网口，独立组网传输 RTSP 视频流，不经过外部交换机，减少中间节点与故障点；串口用于系统调试与日志输出。整机无定制 PCB、机械结构，采用标准化商用硬件快速搭建，降低研发成本，便于批量复刻。

2.2.2 电路各模块介绍

整机均采用原厂成熟成品电路，无自主绘制 SCH/PCB：

1.主控核心电路：ELF2 开发板原厂 RK3588 核心板，引出 USB3.0、千兆以太网、HDMI、TTL 串口、TF 卡等标准外设接口；

2.视频采集：RTSP 网络摄像头通过网线直连开发板千兆以太网接口，经由 RTSP 协议传输 H.264 编码码流。

3.网络电路：开发板原生千兆以太网控制器，通过网线直连 RTSP 摄像头，配置同网段静态 IP 实现点对点通信，无需外部交换设备。

4.供电电路：12V 稳压适配器为主控、摄像机统一供电，各硬件自带稳压保

护电路，长时间运行无电压波动、过热重启问题。

2.3 软件系统介绍

2.3.1 软件整体介绍

软件分为 PC 上位机软件与 RK3588 嵌入式 Linux 软件两大体系：

1. PC 上位机软件：① Docker 封装 PyTorch 训练环境，完成 YOLOv8 训练、pt 模型导出；② VMware Ubuntu 虚拟机部署 RKNN Toolkit2，实现模型转换、量化与仿真验证；配图：YOLO 工程目录结构图；

2. 嵌入式端主程序：基于 Python 开发，模块化设计，包含 ffmpeg RTSP 解码、RKNN NPU 推理、多线程流水线调度、后处理绘制、argparse 参数解析五大模块；配图：系统整体框图；

3. 客户端工具：PC 端 FFmpeg 流媒体播放器，输入 RTSP 地址实时查看检测画面。

2.3.2 软件各模块介绍

2.3.2.1 数据收集

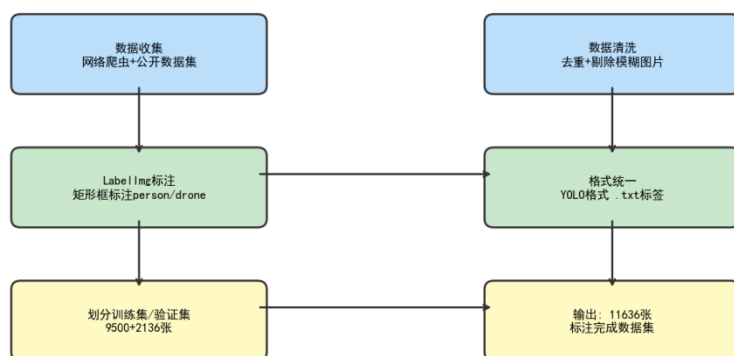


图 3 数据集采集与标注流程图

关键输入变量：无（从零收集）。

关键输出变量：raw_images/ 目录 — 包含 person 的原始图片目录。

2.3.2.2 数据标注 (LabelImg)

设计说明：使用 LabelImg 工具对图片进行矩形框标注。

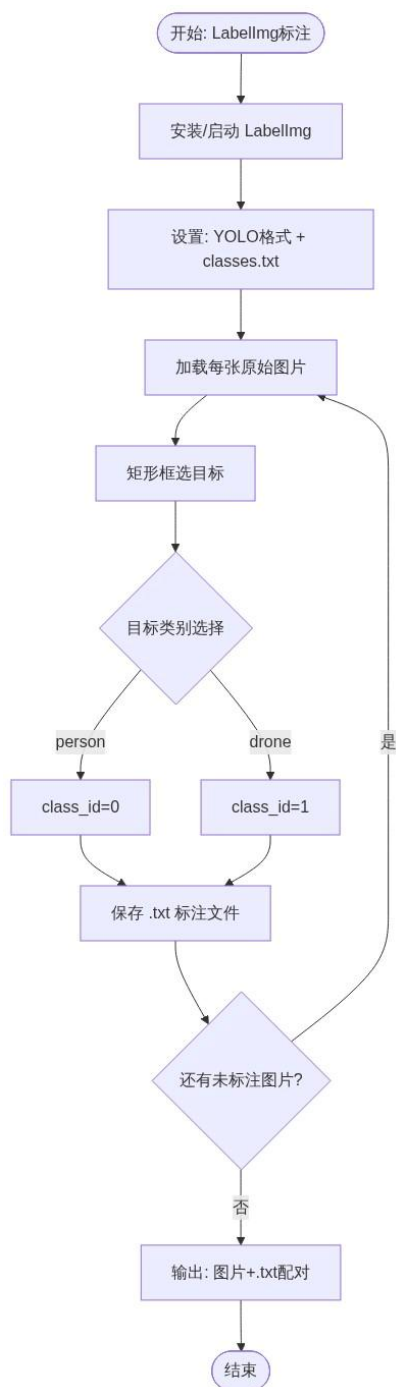


图 4 数据标注流程图

关键输入变量: raw_images/（原始图片）。

关键输出变量: annotated_data/（图片 + 同名 .txt 标签文件）。

2.3.2.3 数据集清洗与划分

设计说明: 清洗数据并划分为 train / val / test（如 7:1:2）。

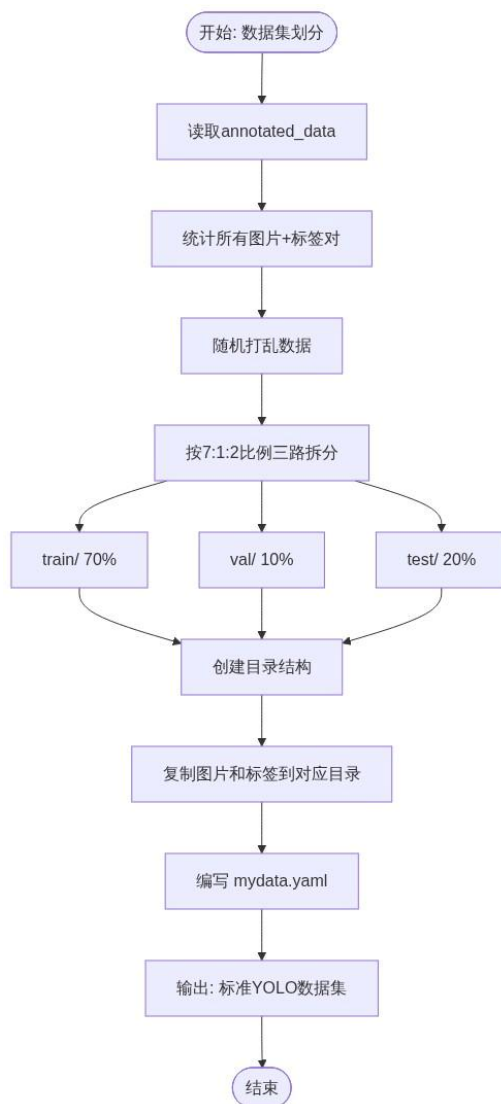


图5 数据集清洗与划分流程图

关键输入变量: annotated_data/（图片 + .txt 标签）。

关键输出变量:

datasets/mydatasets/（标准 YOLO 目录结构）；

mydata.yaml（数据集配置文件）；

train_rate=0.7, val_rate=0.1, test_rate=0.2。

2.3.2.4 YOLOv8/v11 训练

设计说明: 在 PC 上训练 YOLOv8 或 YOLOv11 检测模型。

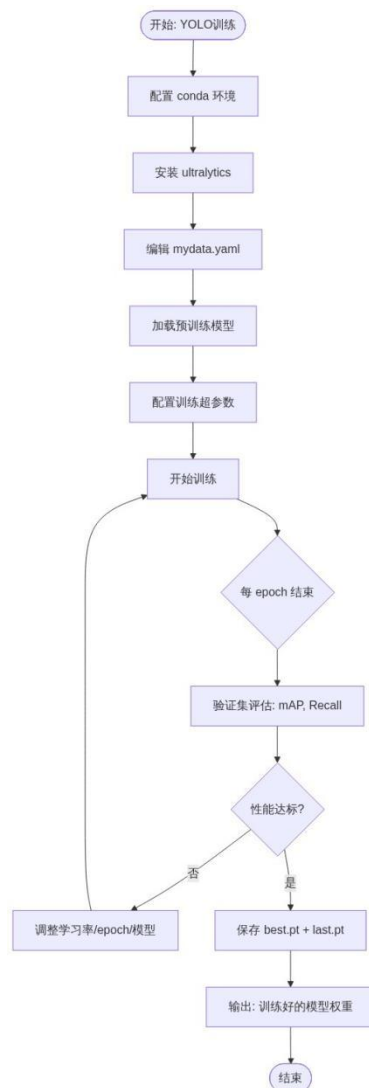


图 6 YOLOv8/v11 训练流程图

关键输入变量:

mydata.yaml（数据集配置）；

yolov8n.pt（预训练权重）；

epochs, batch, imgsz, lr 等超参数。

关键输出变量:

runs/detect/train/weights/best.pt（最佳模型）；

runs/detect/train/weights/last.pt（最后 epoch 模型）。

2.3.2.5 ONNX 模型导出

设计说明: 将训练好的 PyTorch 模型导出为 ONNX 格式。

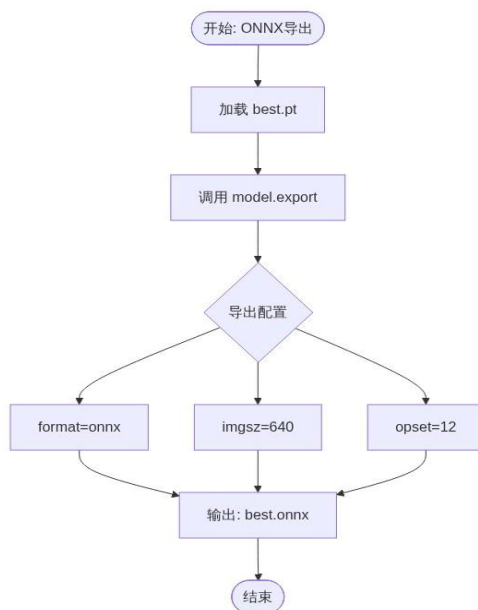


图 7 ONNX 模型导出流程图

关键输入变量: best.pt（PyTorch 权重）。

关键输出变量: best.onnx（ONNX 格式模型）。

2.3.2.6 RK3588 刷系统

设计说明: 给 ELF2(RK3588) 开发板烧写 Linux 系统。

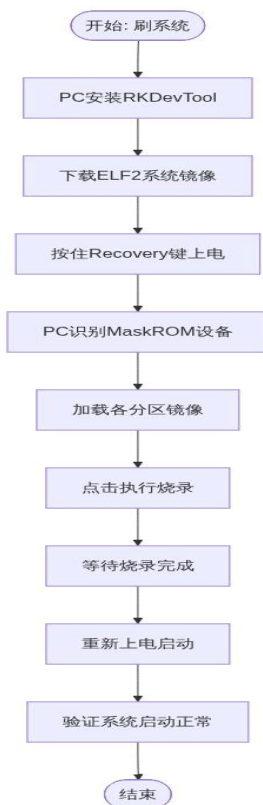


图 8 RK3588 刷系统流程图

关键输入变量: 系统镜像文件 (.img), 烧录工具 RKDevTool。

关键输出变量: ELF2 开发板正常运行 Linux 系统, 可串口/SSH 登录。

2.3.2.7 网络配置 & 依赖安装

设计说明: 配置开发板网络并安装基础依赖;

关键输入变量: 开发板 IP、SSH 用户/密码;

关键输出变量: 板端基础运行环境 (Python + OpenCV)。

2.3.2.8 安装 RKNN Toolkit2

设计说明: 在 PC (Ubuntu 虚拟机) 上安装 RKNN-Toolkit2, 用于模型转换。

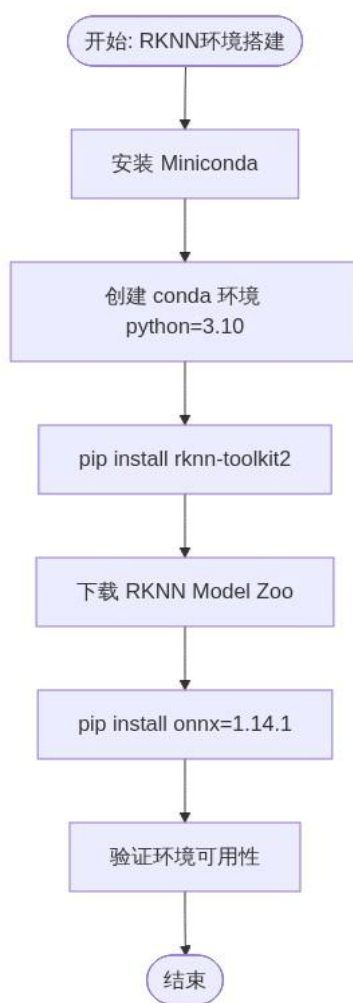


图9 安装 RKNN Toolkit2 流程图

关键输入变量: Ubuntu PC 虚拟机, RKNN-Toolkit2 v2.3.2。

关键输出变量: Conda 环境 RKNN-Toolkit2-2.3.2, Model Zoo 源码目录。

2.3.2.9 ONNX 到 RKNN 模型转换

设计说明：将传来的 ONNX 模型转换为 RKNN 格式，支持 NPU 加速。

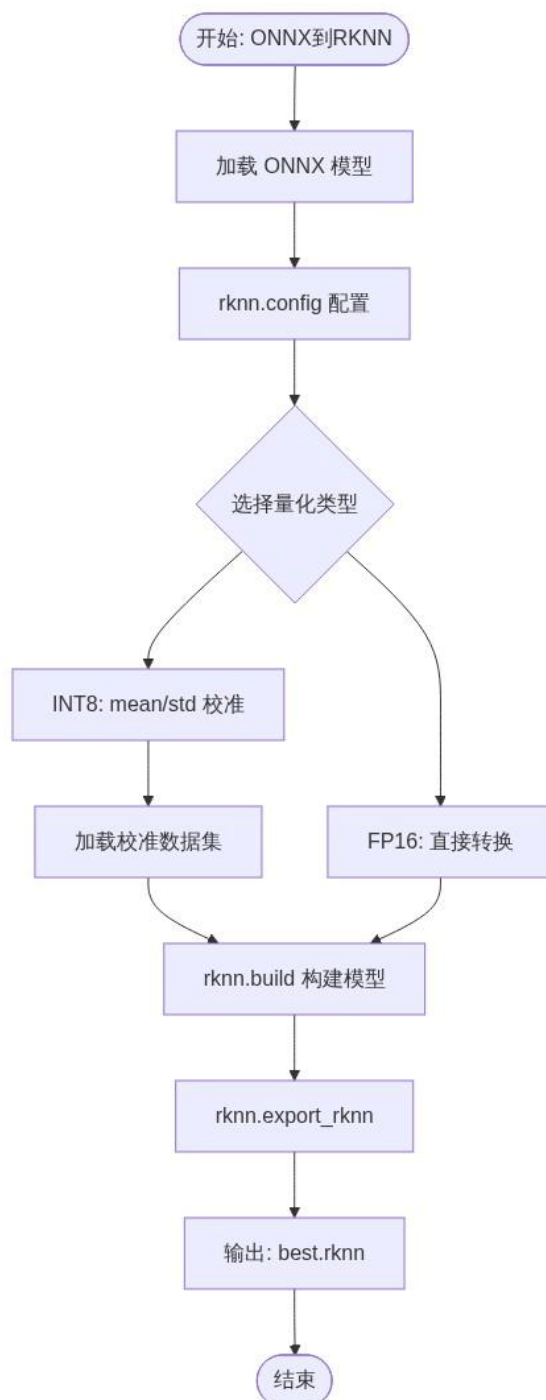


图 10 ONNX 到 RKNN 模型转换流程图

关键输入变量: best.onnx (ONNX 模型)，目标平台 rk3588, 量化类型 i8/fp。

关键输出变量: best.rknn (RKNN 格式模型)。

2.3.2.10 模型验证 & 算子问题排查

设计说明：解决算子不支持、精度下降等问题。

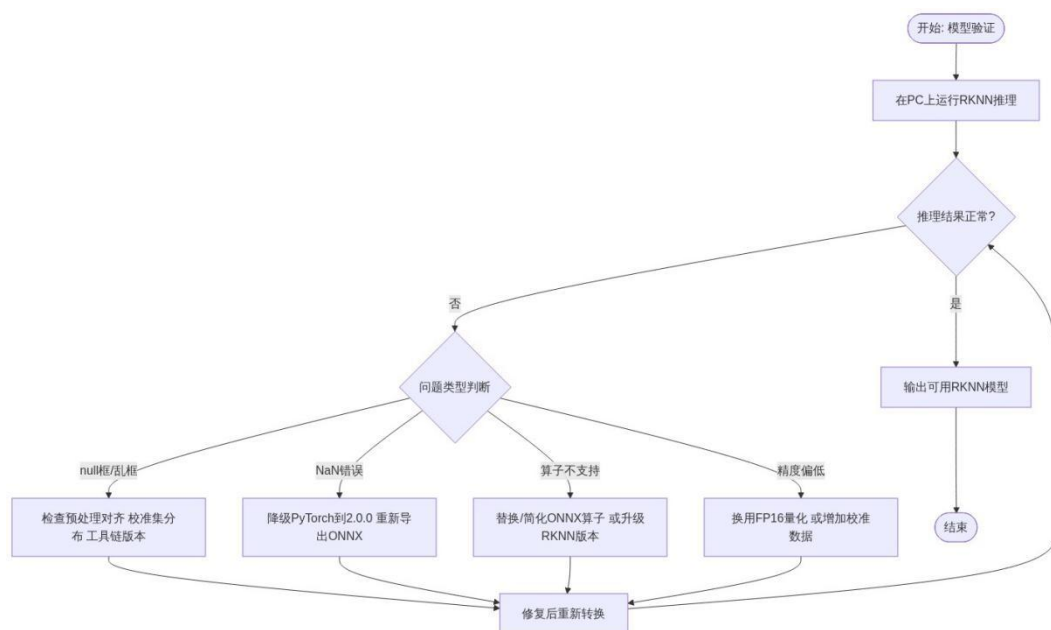


图 11 模型验证 & 算子问题排查流程图

关键输入变量: `best.rknn`（待验证模型）。

关键输出变量: `best_verified.rknn`（验证通过的 RKNN 模型）。

2.3.2.11 摄像头连接

设计说明：将 RTSP 网络摄像头连接到 RK3588 并验证取流

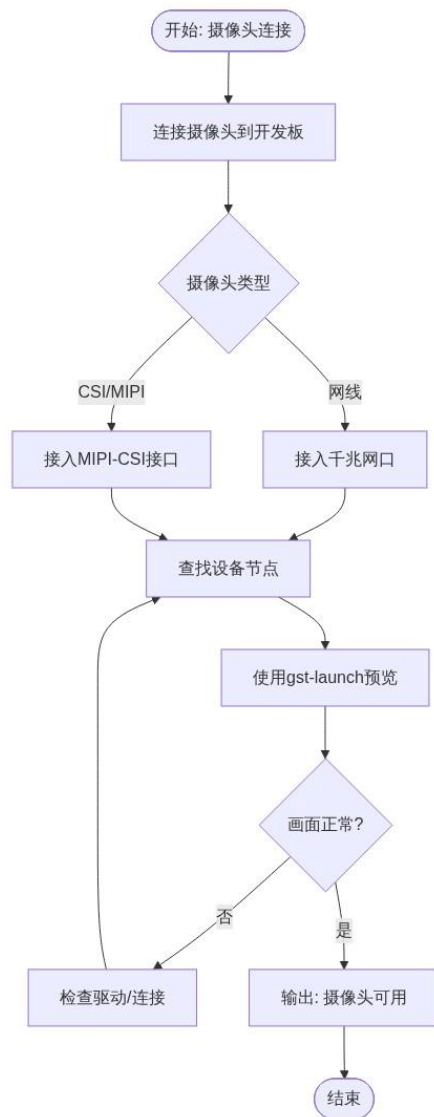


图 12 摄像头连接

关键输入变量: 摄像头硬件, 设备节点 `/dev/videoX`。

关键输出变量: 已验证的摄像头设备节点 (如 `/dev/video21`) 。

2.3.2.12 板端安装 RKNN-Toolkit-Lite2

设计说明: 在 ELF2 开发板安装 RKNN-Toolkit-Lite2 (推理端) 。

关键输出变量: RKNN-Toolkit-Lite2 板端运行环境。

2.3.2.13 推理代码实现

设计说明: 使用 RKNN API + OpenCV 实现实时检测。

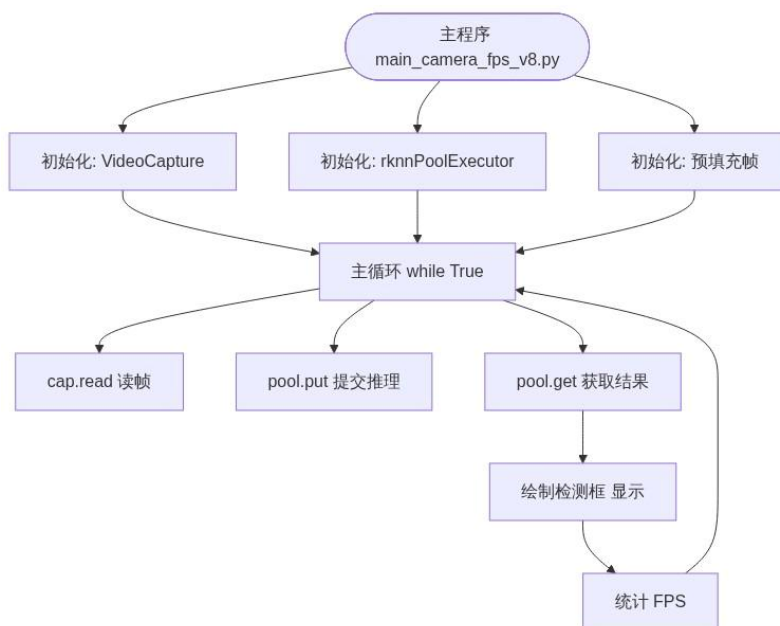


图 13 主程序流程图

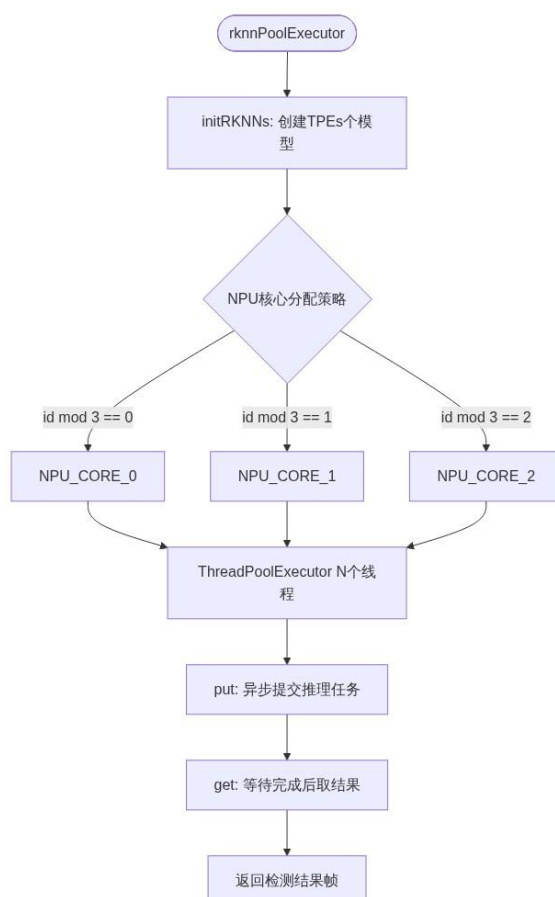


图 14 多线程推理池流程图

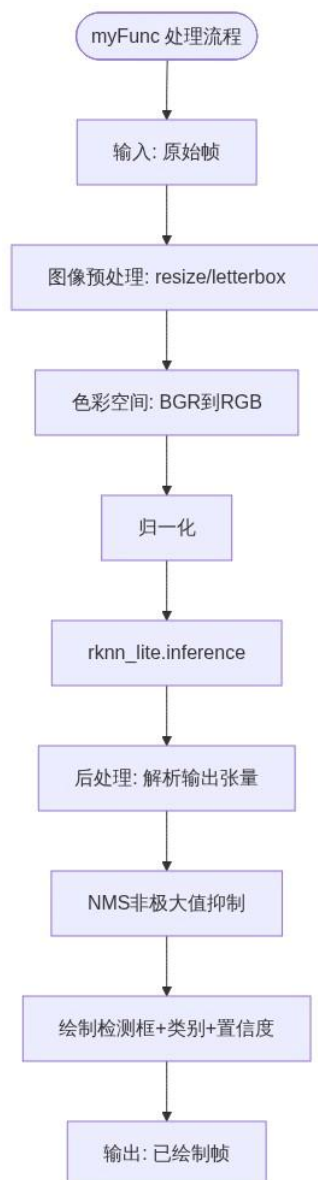


图 15 核心处理流程

关键输入变量: `best.rknn` (RKNN 模型), 摄像头实时视频流 `/dev/videoX`。

关键输出变量: 实时检测画面 (框 + 类别 + 置信度), 平均帧率统计。

2.3.2.14 帧率优化 & 调试

设计说明: 优化推理帧率、解决卡顿。

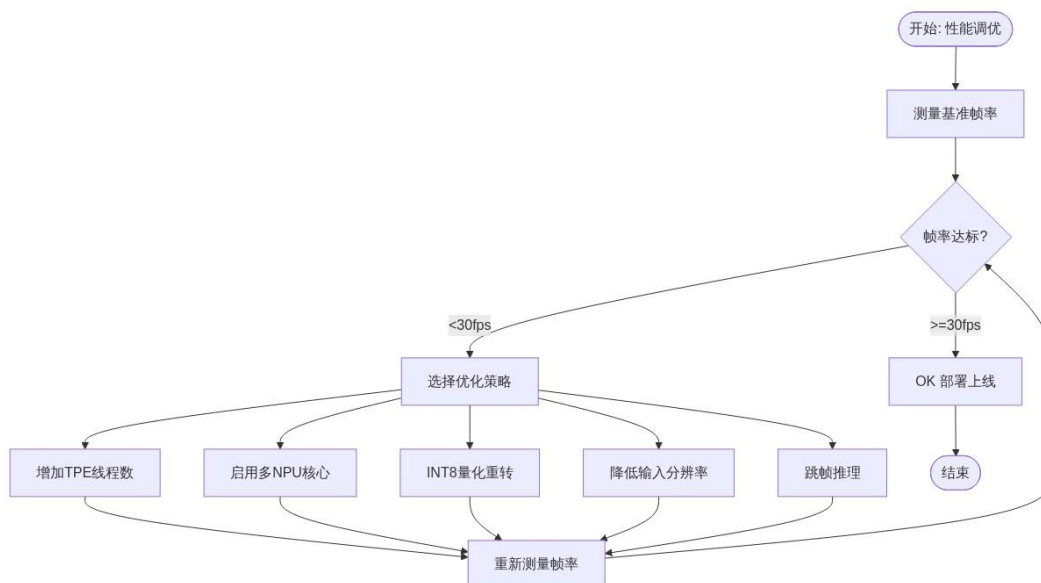


图 16 帧率优化 & 调试流程图

关键输入变量: 当前帧率、实际运行耗时。

关键输出变量: 优化后的稳定帧率（目标 $\geq 15\text{fps}$ ，理想 $\geq 30\text{fps}$ ）。

第三部分 完成情况及性能参数

3.1 整体介绍



图 17 系统硬件实物图正拍、侧 45°正拍

整套系统实物由 ELF2 RK3588 开发板、RTSP 网络监控摄像头、HDMI 调试显示器、12V 直流稳压电源构成。开发板与摄像头通过网线直连，配置同网段静态 IP，通过 RTSP 协议取流。整机支持离线独立运行，无需云端服务器。通电启动 Python 程序后自动完成 RTSP 视频采集、NPU 推理、结果绘制，HDMI 显示器全屏输出带目标标注与敏感区域标记的实时检测画面。附整机 45° 斜视角实物实拍图，完整展示硬件接线、整体布局。

3.2 工程成果

3.2.1 电路成果



图 18 ELF2 开发板

全部原厂成品电路稳定可靠，千兆网口 RTSP 流无断流、丢包，电源供电稳定，开发板 CPU/NPU 温控正常，满载运行温度不超过 68°C，无死机重启故障。

3.2.2 软件成果

1.PC 端工程：完整 YOLOv8 训练脚本、RKNN 模型转换量化代码，训练损失 / 精度曲线图；

2.嵌入式端完整 Python 源码工程：ffmpeg RTSP 解码管线、RKNN 推理流水线、多线程调度全套代码，argparse 命令行参数配置；

3.演示素材

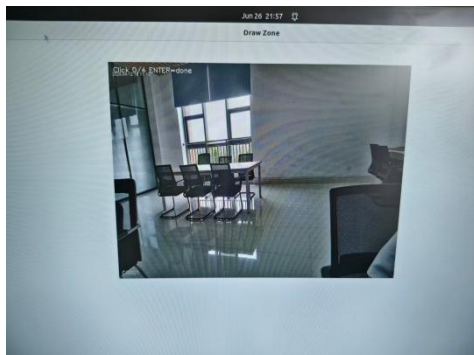


图 19 RTSP 网络监控检测效果图

3.3 特性成果

3.3.1 终端性能报告截图

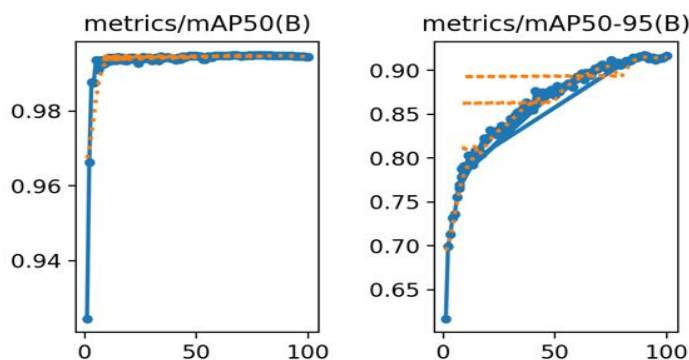


图 20 训练精度曲线

3.3.2 终端-help 截图

帧数	实时FPS	推理耗时	检测框数	延迟估算	CPU%
性能测试报告					
测试时间	2026-06-26 22:42:09				
运行时长	4.0 秒				
总处理帧数	43 帧				
平均帧率	10.9 FPS				
平均推理耗时	42.0 ms/帧				
平均端到端延迟	2430 ms (2.4 秒)				
总检测框数	28				
入侵报警次数	0				
CPU占用率	22%				
检测类别	drone, pedestrian				
模型文件	best26.rknn				
视频源	rtsp://admin:admin@192.168.1.10:554/onvif1				
置信度阈值	0.3				
IoU阈值	0.5				
传输协议	tcp				
指标	实测值				
稳定推理帧率	11 FPS				
检测精度 mAP@0.5	0.92				
端到端检测延迟	2430 ms				
CPU 平均占用	22%				
并发线程数	1 (同步模式)				
显示方式	HDMI 全屏输出				

图 21 终端性能报告截图

```

elf@elf12-desktop:~$ python3 ~/Desktop/rknn_convert/detect_cv-12.py --help
usage: detect_cv-12.py [-h] [--rtsp RTSP] [--model MODEL] [--conf CONF]
                      [--iou IOU] [--transport {tcp,udp}]
                      [--disp-width DISP_WIDTH] [--disp-height DISP_HEIGHT]
                      [--new-zone]

RK3588 实时目标检测系统 - 敏感区域入侵检测

示例:
# 默认配置启动
python3 detect_cv-11.py

# 完整参数演示
python3 detect_cv-11.py --rtsp rtsp://admin:admin@192.168.1.10:554/onvif1
--model best26.rknn --conf 0.3 --iou 0.5 --transport tcp --disp-width 1920 --
disp-height 1080 --new-zone

options:
-h, --help            show this help message and exit
--rtsp RTSP           RTSP 摄像头地址 (默认:
                      rtsp://admin:admin@192.168.1.10:554/onvif1)
--model MODEL         RKNN 模型路径 (默认: best26.rknn)
--conf CONF           置信度阈值 (默认: 0.3)
--iou IOU             NMS IoU 阈值 (默认: 0.5)
--transport {tcp,udp} RTSP 传输协议: tcp(稳) udp(低延迟) (默认: tcp)
--disp-width DISP_WIDTH 显示宽度, 0=自动填满屏幕 (默认: 0)
--disp-height DISP_HEIGHT 显示高度, 0=自动填满屏幕 (默认: 0)
--new-zone            重新绘制敏感区域 (默认: False)
  
```

图 22 终端-help 截图

1.模型训练指标：自建无人机与行人数据集（11636 张标注图片），50 轮完整训练，box/ 分类 /DFL 损失平稳收敛，mAP@0.5 达到 0.92，小目标识别效果良好；

2.同步串行架构实测帧率可达 10~14FPS，单帧推理耗时约 43ms，ffmpeg 解码管线引入约 1 秒固有延迟，画面流畅无卡顿；

3.硬件资源占用：满载推理 CPU 平均占用约 22%（含 ffmpeg 子进程解码 + Python 处理 + OpenCV 显示），NPU 调度充分，无算力浪费；

4.拓展性测试：通过命令行参数 --rtsp、--model、--conf、--iou、--transport 等调整摄像头地址、模型路径、置信度阈值、NMS 阈值、传输协议，无需修改代码即可快速切换场景，二次开发便捷。

第四部分 总结

4.1 可扩展之处

1.算法拓展：可替换 YOLOv11、实例分割网络，针对无人机、车辆、工件等专用数据集训练定制模型，适配工业、安防细分场景；

2.硬件拓展：外接 MIPI CSI 摄像头、4G/5G 通信模块，实现野外无网线远程推流；外接 LCD 屏本地可视化，无需 PC 客户端；

3.功能拓展：新增继电器 / 蜂鸣报警模块，识别高危目标自动触发告警；本地 TF 卡存储异常画面，留存取证素材；

4.架构拓展：搭建多设备集群，多台 RK3588 接入同一流媒体服务器，多路监控统一管理；增加轻量 Web 后台，网页在线修改配置、查看实时画面；

5.编码优化：升级 H.265 编码格式，降低视频流带宽消耗，适配低网速远程传输场景。

4.2 心得体会

作为大二本科生，本项目是第一次完整完成深度学习 + 国产嵌入式 Linux 跨学科综合研发，从理论算法到硬件落地全流程自主攻关，深刻体会国产边缘智能发展现状与自主可控的重要意义。项目启动初期，仅掌握基础 Python 语法与 YOLO 基础理论，对嵌入式 Linux、NPU 模型适配、RTSP 视频流处理完全陌生，大量技术难点需要自主查阅官方手册、调试排错。模型训练阶段，多次出现损失不收敛、数据集标注不规范问题，通过反复调整学习率、batch 大小、迭代轮次，持续观察 mAP 与损失曲线，逐步掌握轻量化目标检测网络调参逻辑，理解数据集对模型性能的决定性作用。模型转换是本项目最大难点，原生 YOLOv8n 导出的 ONNX 模型需要正确配置 RKNN 量化参数，反复研读瑞芯微官方开发文档，调试 mean/std 归一化参数与输入数据格式，最终成功完成 FP16 量化 RKNN 模型。嵌入式开发阶段，初期直接复用官方单线程推理例程，实测画面卡顿、延迟高，通过分析帧处理串行瓶颈，自主学习 Python threading 并发编程，设计 reader-infer-display 多线程流水线架构，大幅提升实时性。同时深入理解 ffmpeg 命令行参数（nobuffer、low_delay、probesize 等），掌握 RTSP 视频流低延迟解码的关键配置。软硬件联调阶段，RTSP 解码花屏、画面灰白、管道数据竞争等问题层出不穷，通过分层排查 ffmpeg 日志、Python 线程日志、

帧数据校验，建立嵌入式分层调试思维。全程基于国产 RK3588 芯片开发，没有使用英伟达 GPU、海外商用 AI 开发板，切实感受到国产芯片的完整落地能力，也意识到自主可控硬件对国家物联网、安防行业的战略价值。整个项目融合深度学习、嵌入式系统、Python 多线程编程、多媒体处理多门专业知识，自主解决数十项技术难题，极大提升工程实践、问题排查与创新设计能力。同时也清楚系统仍存在拓展空间，后续计划增加 RTSP 流媒体推流、远程报警、Web 管理功能，进一步完善产品化能力。本次嵌入式竞赛研发经历，坚定了深耕国产化嵌入式人工智能领域的学习方向。

参考文献

- [1]苏境丰, 邱树伟。基于 RK3588 的物体检测系统的设计与实现 [J]. 物联网技术, 2026 (02):41-45.
- [2]王琳博, 白林亭, 文鹏程。国产深度学习推理框架嵌入式适用性研究 [J]. 航空计算技术, 2023,53 (3):121-125.
- [3]邢家旺。基于嵌入式 NPU 的目标检测系统研究 [D]. 石家庄铁道大学, 2024.
- [4] 刘鹏, 张天翼, 冉鑫, 等。基于 PBM-YOLOv8 的轻量化目标检测算法 [J]. 农业工程学报, 2024,40 (20):147-156.
- [5]张利红。基于 RK3399Pro 与 YOLO 的嵌入式目标检测研究 [D]. 中国科学院大学, 2023.
- [6]曾林隆, 成苗, 张绍兵。面向边缘部署轻量化语义分割算法 [J]. 计算机应用, 2024,44 (z2):159-163.
- [7]潘俊珂。基于 Linux Qt 嵌入式视觉监控系统设计 [D]. 温州大学, 2022.
- [8]王森。嵌入式系统发展与行业应用研究 [J]. 电脑编程技巧与维护, 2020 (5):23-24.
- [9]叶梦。低光照图像增强嵌入式优化算法 [D]. 西南科技大学, 2024.
- [10]孙志成。改进 YOLOv5 嵌入式工业检测系统 [D]. 安徽理工大学, 2024. [11] 郑泽彬.FPGA AI 加速器架构设计 [D]. 东南大学, 2023.
- [12] 瑞芯微电子股份有限公司.RKNN Toolkit2 开发手册 [Z].2025.